

Project Proposal: Reliable Data Transfer Protocol over UDP

Joe Jimenez, Alan Lam
CS 5470 Computer Networks

April 6, 2026

Project Scope

The goal of this project is to implement a reliable data transfer protocol on top of UDP using Python. UDP on its own does not guarantee delivery, ordering, or data integrity, so we will be building all of these reliability features ourselves at the application layer. The idea is to take the core concepts we have studied in class and put them into practice by constructing a working protocol from the ground up.

Our protocol will include the following mechanisms. First it will include checksums to detect the corrupt packets. It will also have ACKs to detect lost and corrupt packets. Sequence numbers will be used for detecting duplicate and lost packets. Timers will be used for handling lost and corrupted packets through retransmission. Retransmission for recovering from lost or corrupted packets. Sliding window for sending and receiving multiple packets and selective repeat so that only the specific lost or corrupted packets get transmitted.

Plan for Implementation

We will write the project in Python using the `socket` library for UDP communication. The system will be split into a sender and a receiver, each running as a separate program. We plan to build the protocol in stages. First, we will get a basic stop and wait version working with checksums, ACKs, sequence numbers, and retransmission on a timer. Then we will expand it to support a sliding window with selective repeat so that multiple packets can be in flight at once. We will also define a custom packet header that carries the necessary fields like sequence numbers, ack numbers, checksums, and flags.

Testing and Evaluation Methods

We plan to test the protocol in two setups. The first setup will run the sender and receiver on the same machine using two separate terminals communicating over localhost. By doing this, it will let us check that everything is working correctly during development. The second setup will run the sender and receiver on two different machines on the same network to test under more realistic conditions.

To test correctness, we will add test cases to confirm that packets are sent and received in the expected order. Timeout and retransmission will be tested by using different timer durations, sleep durations, and probabilistic send functions.

To evaluate performance, we will measure things like throughput, latency, and how well the protocol recovers from packet loss. We will simulate poor network conditions by introducing artificial packet loss, corruption, and delay, and then observe how the protocol handles it. We will also experiment with different window sizes to see how they affect performance.